

AI-IMU++: Dead-Reckoning Accounting for Centrifugal Acceleration

Xuan Phuc Nguyen
Skolkovo Institute
of science and technology
Moscow, Russia
Xuan.Nguyen@skoltech.ru

Maksim Alymov
Skolkovo Institute
of science and technology
Moscow, Russia
Maksim.Alymov@skoltech.ru

Abstract—Traditional solutions of Dead Reckoning problem suffer from error accumulation, which make them not suitable for situations, where accurate state estimation is crucial. In this report we describe work where we tried to solve such problem. As the baseline we took [1] paper. We introduce new measurements model, which goal is to avoid variance growth in the maneuvers.

<https://github.com/CorporalCleg/ai-imu-dr>

Index Terms—Localization, Navigation, State Estimation, Artificial Intelligence

I. INTRODUCTION

A. Problem

Inertial navigation is a popular solution for state estimation. It is utilized in robotics, autonomous vehicles, photogrammetry and any other domain, where pose estimation is necessary. However such sensors accumulate error rapidly, which makes IMU-only navigation system very inaccurate.

Modern solutions use different tricks to improve basic approach. One of them is to utilize dynamics constraints as pseudo-measurements. Also popular solution is to use ai-models, which find sensors patterns to correct inaccurate estimations.

II. ORIGINAL APPROACH

A. Dead reckoning problem

Essentially dead reckoning is a navigation problem, where given observations $\{z_i\}_{i=1}^N$ the task is to estimate state vector $\hat{x}_i$

On our case, the estimated state:

$$X_n = (R_n^{\text{IMU}}, v_n^{\text{IMU}}, p_n, b_n^a, b_n^\omega, R^{\text{car}}, p^{\text{car}}) \quad (1)$$

With given observations:

$$\{a_i, \omega_i\}_{i=1}^N \quad (2)$$

where a_i, ω_i — IMU measurements in i -th step

B. Invariant EKF

a) Propagation model:

$$\omega_n^{\text{IMU}} = \omega_n + b_n^\omega + w_n^\omega \quad (3)$$

$$a_n^{\text{IMU}} = a_n + b_n^a + w_n^a \quad (4)$$

where

$$b_n^\omega, b_n^a \quad (5)$$

are quasi-constant biases and

$$w_n^\omega, w_n^a \quad (6)$$

are zero-mean Gaussian noises. The biases follow a random walk

$$b_{n+1}^\omega = b_n^\omega + w_n^{b_\omega} \quad (7)$$

$$b_{n+1}^a = b_n^a + w_n^{b_a} \quad (8)$$

where

$$w_n^{b_\omega}, w_n^{b_a} \quad (9)$$

are zero-mean Gaussian noises.

The kinematic model is governed by the following equations

$$R_{n+1}^{\text{IMU}} = R_n^{\text{IMU}} \exp((\omega_n \, dt) \times) \quad (10)$$

$$v_{n+1}^{\text{IMU}} = v_n^{\text{IMU}} + (R_n^{\text{IMU}} a_n + g) \, dt \quad (11)$$

$$p_{n+1}^{\text{IMU}} = p_n^{\text{IMU}} + v_n^{\text{IMU}} \, dt \quad (12)$$

$$R_{n+1}^c = R_n^c \exp((w_n^{R_c})_\times) \quad (13)$$

$$P_{n+1}^c = P_n^c + w_n^{P_c} \quad (14)$$

For more details, you can look at the original paper. Just be careful that some formulas (number (5), (12), (37)) in [1] is not correct. The derivation process and the H, F matrices were introduced in [2].

b) Pseudo-Measurement model:

To develop and self-calibrate a filter, we use only an IMU, without additional devices like cameras, lidars, or GPS. The approach of leveraging physical properties (kinematics and dynamics) to constrain the state, serving as a measurement step in filtering techniques, is referred to as pseudo-measurements.

In the original paper, the author considers a car with a single IMU sensor rigidly attached to it. With the reference coordinate system chosen in the IMU frame, the velocity of the car in the world frame can be expressed as:

$$v_n^c = (v_n^{\text{for}} \ v_n^{\text{lat}} \ v_n^{\text{up}})^T = R_n^{cT} (R_n^{\text{IMUT}} v_n^{\text{IMU}} + (\omega_n) \times p_n^c) \quad (15)$$

This equation essentially transforms the car's velocity, initially expressed in the IMU frame, into the car frame using the estimated rotation matrix R_n^{cT} .

Since the car can only move forward or backward, it cannot have lateral or vertical velocity. Therefore, we impose the constraints $v_n^{\text{lat}} = 0$ and $v_n^{\text{up}} = 0$. However, to integrate these constraints within a Kalman filter framework, they must be softened—rather than enforcing strict equality, we introduce some noise into the equation. This gives us:

$$y_n = [h^{\text{lat}}(x_n) + n_n^{\text{lat}}, h^{\text{up}}(x_n) + n_n^{\text{up}}]^T = [v_n^{\text{lat}}, v_n^{\text{up}}]^T + n_n^T \quad (16)$$

c) AI integration:

Since pseudo-measurements depend on the system's kinematic and dynamic properties, as well as the system's state (which is inherently linked to sensor outputs), accounting for all influencing factors in the observation function becomes increasingly complex. To address this, the author proposes using a neural network to estimate the covariance matrix.

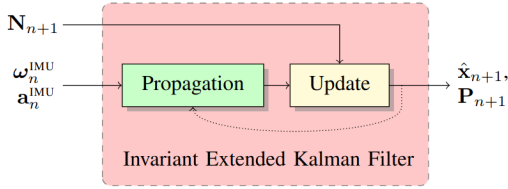


Fig. 1: IEKF uses IMU - measurements for propagate step and predicted pseudo-measurements covariance matrix in update

While this method is reasonable, it introduces a drawback: the covariance now depends on the specific physical characteristics of the IMU, making generalization difficult.

Another problem is that the pseudo-measurement for lateral velocity was ignored when the car began turning, most likely due to centrifugal acceleration during the turning process. The similar situation as occurred with pseudo measurement of vertical velocity. But may be it can be interpreted as the suspension system and noise of non-plat road impact IMU

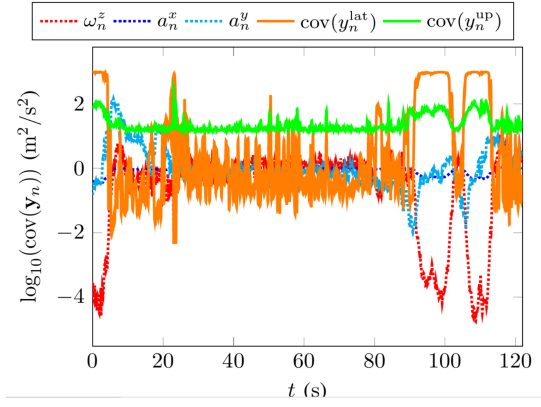


Fig. 2: The car turning, covariance of Pseudo measurements went to unsearable big value

III. PROPOSED SOLUTION

A. New measurement model

Our idea is to introduce an additional constraint on lateral acceleration along the y-axis, convert it into a new pseudo-measurement, and retrain the neural network to estimate three variance values.

Using the Ackermann model, a four-wheeled car can be approximated as a bicycle model [3].

The additional pseudo-measurement constraint states that the car's lateral acceleration should equal the centrifugal acceleration:

$$h(X) = a_y^{\text{car}} - \nu^{\text{for}} \cdot \omega_z^{\text{car}} \quad (17)$$

a) Components Breakdown:

For a_y^{car}

a^{car} is the acceleration of the car's body with respect to the global frame O , but expressed in the car's frame.

$a_{\frac{B}{O}}$	$a_{\frac{A}{O}}$	$\dot{\omega} \times r_{AB}$	$\omega \times (\omega \times r_{AB})$
a^{car}	$R_n^{\text{car}T}(a_n)$	$R_n^{\text{car}T}(\dot{\omega}_n \times p_n^c)$	$R_n^{\text{car}T}(\omega_n \times (\omega_n \times p_n^c))$
Explanation	“Acceleration transformation from IMU to car frame”	“Tangent acceleration of car around IMU”	“Centripetal (centrifugal) acceleration term”

Assumption:

$$\dot{\omega}_{\text{IMU}} \times p_n^c = 0 \quad (18)$$

(valid when the car isn't rotating too fast). Thus:

$$a^{\text{car}} = R_n^{\text{car}T}((a_n) + \omega_n \times (\omega_n \times p_n^c)) \quad (19)$$

The lateral component is:

$$a_y^{\text{car}} = (0, 1, 0) a^{\text{car}} = M_1 R_n^{\text{car}T} \alpha_1$$

For ν^{for} The forward velocity component is:

$$\begin{aligned} v^{\text{for}} &= (1, 0, 0) R_n^{\text{car}T} (R_n^{\text{IMUT}} v_n^{\text{IMU}} + \omega_n^c \times p_n^c) \\ &= M_2 R_n^{\text{car}T} \alpha_2 \end{aligned} \quad (20)$$

For ω_z^{car} Since the car and IMU are fixed together, they rotate as one body:

$$\omega^{\text{car}} \& \text{imu} = \omega_n = \omega_n^{\text{IMU}} - \beta_n^\omega \quad (21)$$

Accounting for frame misalignment, the yaw rate in the car's frame is:

$$\omega^{\text{car}} = (0, 0, 1) R_n^{\text{car T}} (\omega_n^{\text{IMU}} - \beta_n^\omega) = M_3 R_n^{\text{car T}} \alpha_3 \quad (22)$$

Here is our new Jacobian matrix (see Appendix A for more details):

	$R_n^{\{\text{imu}\}}$	$v_n^{\{\text{imu}\}}$	$p_n^{\{\text{imu}\}}$	b_{ω_n}	b_{a_n}	R_n^c	p_n^c
$v_c[1]$	H_{11}	H_{12}	H_{13}	H_{14}	H_{15}	H_{16}	H_{17}
$v_c[2]$	H_{21}	H_{22}	H_{23}	H_{24}	H_{25}	H_{26}	H_{27}
$h(X)$	H_{31}	H_{32}	H_{33}	H_{34}	H_{35}	H_{36}	H_{37}

B. Model training

a) Net achitecture:

In the original original paper the complete architecture of the adapter consists of a bunch of CNN layers followed by a full layer outputting a vector

$$z_n = [z_n^{\text{lat}}, z_n^{\text{up}}]^T \in \mathbb{R}^2 \quad (23)$$

The covariance

$$N_{n+1} \in \mathbb{R}^{2 \times 2} \quad (24)$$

is then computed as

$$N_{n+1} = \text{diag}(\sigma_{\text{lat}}^2 10^{\beta \tanh(z_n^{\text{lat}})}, \sigma_{\text{up}}^2 10^{\beta \tanh(z_n^{\text{up}})}) \quad (25)$$

As we added new observation, we should extend neural net output to predict measurements all measurements.

$$N_{n+1} = \text{diag}(\sigma_{\text{lat}}^2 10^{\beta \tanh(z_n^{\text{lat}})}, \sigma_{\text{up}}^2 10^{\beta \tanh(z_n^{\text{up}})}, \sigma_{\text{acc}}^2 10^{\beta \tanh(z_n^{\text{acc}})}) \quad (26)$$

b) Training process:

Data:

We trained the model on the sequences which are proposed by authors of the original of the original method, which includes sequences

Network architecture: 2-layers 1d CNN

Trainer parameters:

lr: 0.0001 number of epochs: 60

IV. RESULTS AND COMPARISON

a) *Plots*: The figures below we provide estimated trajectory or the original approach and our one

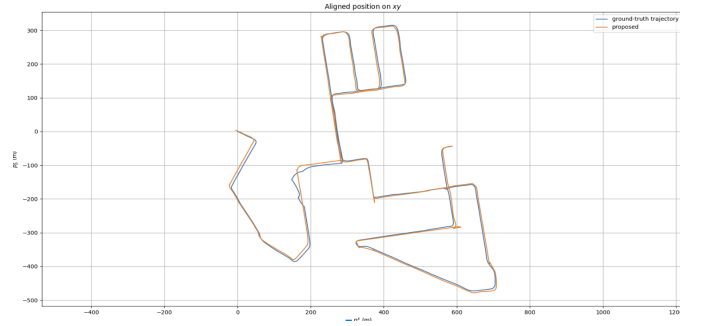


Fig. 3: Original filter prediction

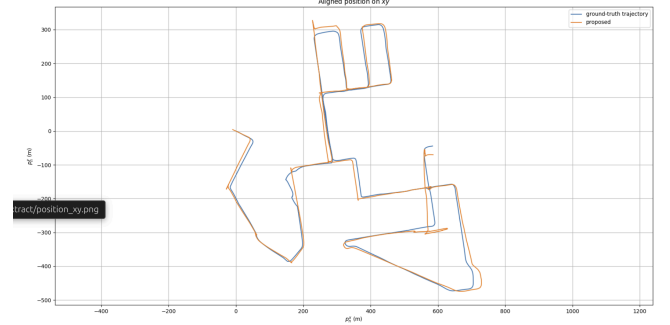


Fig. 4: Ours filter prediction

in the trajectory, which was predicted by our filter still deviates from the ground truth.

Also, in the plots below, we provide data that illustrates the behavior of the predicted covariances. The green line in the first plot represents the predicted covariance over time, which was previously almost ignored. However, now the covariance for lateral velocity has decreased near zero.

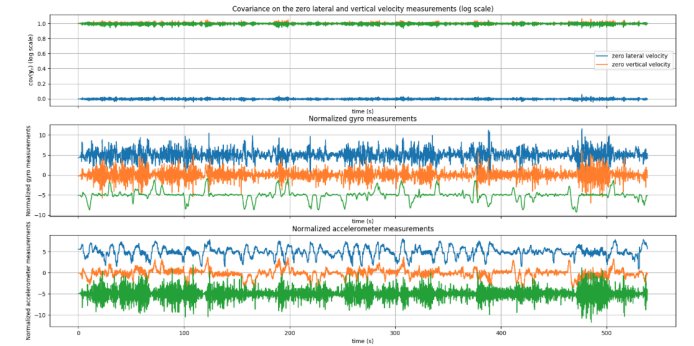


Fig. 5: Predicted covariances plot;

and position and velocities over time

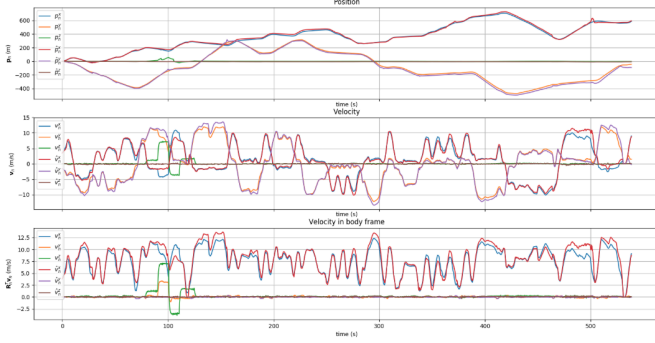


Fig. 6: Predicted covariances plot; the green line in the first plot is predicted covariance over time

b) Analysis:

In the Fig. 4 we see that trajectory deviates from the original one still significantly. In the segment when trajectory rapidly changes its direction filter deviates and for this reason we can observe sharp peaks on this places.

Looking at the Fig. 5 we see the covariances which are predicted by neural net. They are variates near some constant value, which proves, that filter didn't learn what it should to predict, because filter should increase uncertainty (and variance consequently) in the trajectory segment, where the car makes rotation, where centrifugal acceleration exists and decrease it when the vehicle moves along the straight line.

During the training process, we encountered some strange behaviors. The loss function fluctuated, there may still be errors in the problem formulation, making the filter incorrect. Additionally, the results depend on fine-tuning the covariance matrix Q in the propagation step, which requires further trials and testing.

Such result can be explained for some problems, here they are (in the order of importance):

- Potential misalignment between the proposed measurement model and the physical dynamics during maneuvers,
- Suboptimal initialization of covariance parameters for the new pseudo-measurement channel,
- Insufficient neural network training time and data diversity.

V. CONCLUSIONS

In this work, we proposed an enhancement to the AI-IMU dead-reckoning framework by introducing a centrifugal acceleration pseudo-measurement to better account for lateral dynamics during maneuvers. Building on the invariant EKF architecture from [1], our approach extended the neural network to predict covariances for three pseudo-measurements (lateral velocity, vertical velocity, and lateral acceleration) instead of two. While initial results (Section) demonstrated partial trajectory estimation improvements, significant devia-

tions persisted during sharp turns, and covariance predictions did not adapt dynamically to motion patterns as anticipated.

Future work should focus on rigorous covariance parameter calibration, integration of vehicle dynamics models (e.g., Ackermann constraints), and investigation of alternative network architectures for covariance prediction. Despite current limitations, this study highlights the importance of systematic model validation and the challenges of balancing physical constraints with data-driven adaptations in inertial navigation systems.

VI. APPENDIX A JACOBIAN MATRIX

Jacobian derivation:

The formulas in (16) and (17) may appear complex at first glance, but we can break them down and find the Jacobian for each term in the state. For the vector terms $(b_n^a, b_n^\omega, p^{\{car\}})$, the process is relatively straightforward, as we only need to find the corresponding formula in standard vector space.

For the state belonging to a Lie group, we use the general formula for the Lie algebra: [4]:

$$\frac{\Delta f(X)}{\Delta X} = \lim_{\{\tau \rightarrow 0\}} \frac{f(\tau \circ X) - f(X)}{\tau} \in \mathbf{R}^{\{n \times m\}} \quad (27)$$

Now, we need to express our function as a function of the Lie element $\mathbf{R}_c$ for $SO(3)$ and χ for $SE_2(3)$. The term $\mathbf{R}_c$ has already been separated, and for χ , fortunately, we can rewrite the equation in terms of χ^{-1} :

$$\chi_n^{-1} = \begin{pmatrix} R_n^T & -R_n^T v_n & -R_n^T p_n \\ \mathbf{0} & 1 & 0 \\ \mathbf{0} & 0 & 1 \end{pmatrix} \quad (28)$$

Furthermore, the term $R_{imu}^T \cdot v_n$ can be expressed as

$$\chi_n^{\{-1\}} b \quad (29)$$

where

$$b = (0 \ 0 \ 0 \ 1 \ 0)^T \quad (30)$$

And we can simplify our Jacobian matrix to:

$$H_{11} = \mathbf{0}_{\{1 \times 3\}} \quad (31)$$

$$H_{12} = M_1 (R_c^T R_{imu}^T) \quad (32)$$

$$H_{13} = \mathbf{0}_{\{1 \times 3\}} \quad (33)$$

$$H_{14} = M_1 R_c^T (p_c)_\times \quad (34)$$

$$H_{15} = \mathbf{0}_{\{1 \times 3\}} \quad (35)$$

$$H_{16} = M_1 R_c^T (\alpha)_\times \quad (36)$$

$$H_{17} = M_1 R_c^T (\omega_n - b_\omega)_\times \quad (37)$$

$$H_{21} = \mathbf{0}_{\{1 \times 3\}} \quad (38)$$

$$H_{22} = M_3 (R_c^T R_{imu}^T) \quad (39)$$

$$H_{23} = \mathbf{0}_{\{1 \times 3\}} \quad (40)$$

$$H_{24} = M_3 R_c^T (p_c)_\times \quad (41)$$

$$H_{25} = \mathbf{0}_{\{1 \times 3\}} \quad (42)$$

$$H_{26} = M_3 R_c^T(\alpha)_\times \quad (43)$$

$$H_{27} = M_3 R_c^T(\omega_n - b_\omega)_\times \quad (44)$$

$$H_{31} = \mathbf{0}_{\{1 \times 3\}} \quad (45)$$

$$H_{32} = -M_2(R_c^T R_{\text{imu}}^T)\omega_{\text{car}}[2] \quad (46)$$

$$H_{33} = \mathbf{0}_{\{1 \times 3\}} \quad (47)$$

$$H_{34} = M_1 R_c^T[-\omega_n - b_\omega p_c^T - (\omega_n - b_\omega \cdot p_c)\mathbf{I} \\ + 2p_c \omega_n - b_\omega^T] + M_2 R_c^T\{p_c\}_\times \cdot \omega_{n[2]} + M_3 R_c^T \cdot \nu^{\text{for}}$$

$$H_{35} = -M_1 R_c^T \quad (48)$$

$$H_{36} = M_1 R_c^T(\alpha_1)_\times - M_2 R_c^T(\alpha_2)_\times \omega_{\text{car}}[2] \\ - \nu^{\text{for}} M_3 R_c^T(\alpha_3)_\times \quad (49)$$

$$H_{37} = M_1 R_c^T(\omega_n - b_\omega) \cdot (\omega_n - b_\omega)^T \\ - (\omega_n - b_\omega)^T (\omega_n - b_\omega) \mathbf{I} + M_2 R_c^T(b_\omega - \omega_n)_\times \omega_{\text{car}}[2] \quad (50)$$

REFERENCES

- [1] M. Brossard, A. Barrau, and S. Bonnabel, “AI-IMU Dead-Reckoning,” *IEEE Transactions on Intelligent Vehicles*, 2020.
- [2] A. Barrau and S. Bonnabel, “The Invariant Extended Kalman Filter as a Stable Observer.” 2015.
- [3] T. Fermi, “Bicycle Model for Automated Driving Control.” 2025.
- [4] D. A. Joan Solà Jeremie Deray, “A micro Lie theory for state estimation in robotics.” 2015.